# Night Shift

On-call agents that never solve the same incident twice, even when the other shift solved it.

Same answers as a wiki holding every on-call note, at **42% of the tokens**;
**39% fewer** wrong fixes applied, thanks to the Postgres ledger

ContextCon: Remember Everything · Theme 02 Agentic support: Never Solve It Twice

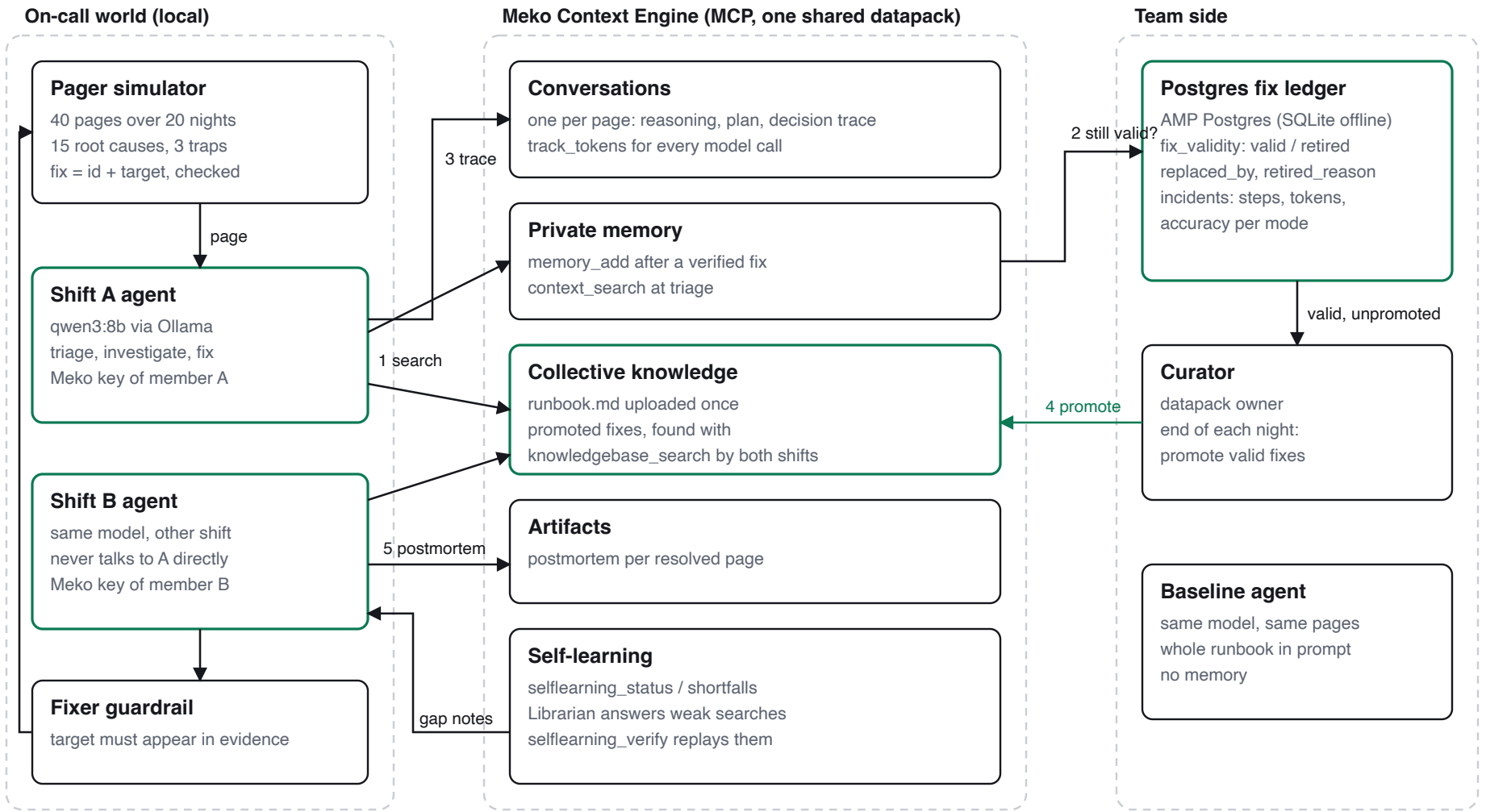
github.com/kangraemin/nightshift-meko

# The same page gets solved twice

- Internal on-call teams split the night into shifts.
- The engineer who fixed the 2 a.m. page writes it down in their own notes, if at all.
- The next shift gets the same page and starts from the wiki again.
- The wiki is half right: it describes last year's causes, not this week's.
- Worse, a remembered fix can stop working. Applying it again wastes the first ten minutes of an outage.

Night Shift turns each verified fix into team memory, and keeps a ledger of which remembered fixes still work.

# How it works


On-call world (local)
Pager simulator
40 pages over 20 nights
15 root causes, 3 traps
fix = id + target, checked
page
Shift A agent
qwen3:8b via Ollama
triage, investigate, fix
Meko key of member A
Shift B agent
same model, other shift
never talks to A directly
Meko key of member B
Fixer guardrail
target must appear in evidence
Meko Context Engine (MCP, one shared datapack)
Conversations
one per page: reasoning, plan, decision trace
track_tokens for every model call
Private memory
memory_add after a verified fix
context_search at triage
Collective knowledge
runbook.md uploaded once
promoted fixes, found with
knowledgebase_search by both shifts
Artifacts
postmortem per resolved page
Self-learning
selflearning_status / shortfalls
Librarian answers weak searches
selflearning_verify replays them
3 trace
1 search
5 postmortem
gap notes
Team side
Postgres fix ledger
AMP Postgres (SQLite offline)
fix_validity: valid / retired
replaced_by, retired_reason
incidents: steps, tokens,
accuracy per mode
2 still valid?
valid, unpromoted
Curator
datapack owner
end of each night:
promote valid fixes
4 promote
Baseline agent
same model, same pages
whole runbook in prompt
no memory

# One page, end to end

- **Triage.** The shift searches its own memory and the team's collective knowledge in Meko.
- **Check.** Every hit is joined with the Postgres fix ledger: valid, retired, or unverified.
- **Investigate.** The agent runs diagnostics. The Fixer only applies a fix to a target seen in evidence or memory.
- **Remember.** A verified fix becomes a private memory, a postmortem artifact and a ledger row.
- **Learn from people.** When the agent gives up, the on-call human's fix is stored the same way, marked as written by a person.
- **Share.** At the end of the night the datapack owner promotes every fix the ledger still marks valid. The other shift finds them next time.
- **Forget on purpose.** When a remembered fix fails, the ledger retires it and the Fixer refuses it from then on.

# Postgres keeps memory honest

Promoted memories cannot be edited or deleted over MCP. Retrieval alone cannot tell a good memory from a stale one. So the ledger decides.

---

```
INC-24 · checkout is throwing 502s, p99 latency 2s
apply_fix F01 pgb-checkout-rw on pgbouncer: REFUSED by Fixer: the fix ledger retired F01 pgb-checkout-rw for this alert (INC-20: F01 applied, symptoms per…
get_pool_stats on pgbouncer: pgbouncer: pool cl_waiting=0 cl_active=3 maxwait=0s
get_config on checkout-api: checkout-api: config unchanged in 15 days
get_queue on notify-queue: notify-queue: no queues owned or depth=0
apply_fix F09 new_pricing_engine on checkout-api: disable_flag <flag name> new_pricing_engine applied on checkout-api: symptoms persist (checkout is throwing 50…
get_metrics on checkout-api: checkout-api: symptom confirmed (checkout is throwing 502s, p99 latency 2s); own cpu/mem normal; errors are on…
get_dependencies on checkout-api: checkout-api upstreams: pgbouncer healthy, postgres DEGRADED (errors/timeouts from here), payments healthy, in…
```

# Same model, same 40 pages, 3 live runs

qwen3:8b on a laptop. Left: whole runbook in the prompt. Right: Night Shift. Graded by a deterministic simulator.

| | Pages | Runbook in prompt | Night Shift | Steps | Tokens per page |
|---|---|---|---|---|---|
| First seen | 12 | 56% | **61%** | 8.4 → 7.2 | 11,173 → 8,948 |
| Repeat | 25 | 56% | **92%** | 8.7 → 2.8 | 11,662 → 3,972 |
| Trap | 3 | 33% | 33% | 9.7 → 9.4 | 13,358 → 13,945 |
| All | 40 | 54% | **78%** | 8.7 → 4.6 | 11,642 → 6,213 |

Mean of 3 live runs (each shuffles which shift gets which page). On repeat pages Night Shift resolved 92% (96% / 92% / 88%) against 56%, with 66% fewer tokens. Most of this jump is the human notes reaching the next shift; the next slide gives a wiki the same notes.

# Where the gain comes from

Same pages and model offline; each row switches one thing.

| Offline lane | First seen | Repeats | Steps per repeat |
|---|---|---|---|
| Runbook pasted | 61% | 55% | 8.8 |
| Runbook pasted + human notes in the wiki | 56% | 91% | 5.9 |
| Runbook search only, no memory | 61% | 65% | 7.8 |
| Private memory only | 67% | 72% | 5.9 |
| Shared, agent fixes only | 64% | **71%** | 5.4 |
| Shared, agent + human fixes | 61% | **89%** | 3.3 |

Offline, same pages, mean of 3 seeds. Human notes are written by the simulator from the true cause. With no human input the agents' own memory lifts repeats 55% → 71%. Given the same notes, a wiki resolves as many repeats (91% vs 89%) but needs 11,366 tokens per repeat against 4,816.

# Does the ledger matter?

Same offline runs with the Postgres ledger switched off: memories are still learned and shared, but every remembered fix looks valid forever.

| Night Shift lane | Repeats | Traps | Failed fixes applied | Fixes refused |
|---|---|---|---|---|
| Memory without the ledger | 91% | 33% | 19.7 per run | 22.3 per run |
| Memory checked against the ledger | 89% | 44% | **12.0 per run** | 32.0 per run |

Offline, mean of 3 seeds. A failed fix is a change to production (rollback, restart, drain, config edit) applied to the simulated production that did not clear the page. Resolution barely moves; wrong changes to production drop 39%. With 10 traps per run (3 seeds): the trap page itself is no easier (40% → 33%), but when the new cause returns it is resolved 20/30 → 24/30, and failed fixes drop 38.7 → 28.3 per run.

# People stay in the loop

- **Review before sharing.** A learned fix stays with its shift until a person approves it (`nightshift.review approve`). A rejected fix is retired, so the Fixer refuses it.
- **Fixes made by hand.** When the agent gives up, the engineer's fix goes into Meko as that shift's memory, marked as written by a person. Over the 3 live runs: 26 human notes, and later repeat pages that started from one were resolved 28 times out of 30.
- **Real alerts.** An Alertmanager webhook runs the same triage on a live alert: what the team already fixed, who fixed it on which page, and which old fix the ledger says not to re-apply. It suggests; it never acts.

# Every Meko layer has a job

| | |
|---|---|
| Conversations + messages | decision trace for every page, reasoning and plan |
| Token tracking | local-model tokens next to each trace |
| Private memory | memory_add after a verified fix, context_search at triage |
| Collective knowledge | runbook upload, memory_promote by the curator, knowledgebase_search |
| Artifacts | postmortem markdown per resolved page |
| Self-learning | shortfalls answered by a librarian, then replayed with selflearning_verify |
| Datapack sharing | shift B is a second Meko member with Contributor access |
| Postgres in the context | fix_validity ledger joined with every memory hit |

# Limits and next steps

- The incidents are simulated so that grading is exact. Real pages are messier.
- One 8B local model, no paid API. A larger model was not tested.
- Traps stay hard: 33% of trap pages resolved, same as the runbook lane (33%). The ledger stops the old fix; finding the new cause is still up to the model.
- Next: wire the agent's diagnostics to real tools behind the webhook, and let the ledger expire fixes after a deploy.

Code, replay and results: github.com/kangraemin/nightshift-meko